

Отчёт по лабораторной работе № 1

Подготовка стенда

Сергей Витальевич Павлюченков

Table of contents

1	Цель работы	5
2	Задание	6
3	Выполнение лабораторной работы	7
4	Параметрическое исследование экспоненциального роста	13
4.1	Активация проекта и загрузка пакетов	13
4.2	Определение модели	14
4.3	Определение параметров в Dict	14
4.4	Функция-обертка для запуска одного эксперимента	15
4.5	Запуск базового эксперимента	16
4.6	Визуализация базового эксперимента	16
4.7	Параметрическое сканирование	17
4.8	Запуск всех экспериментов и сбор результатов	19
4.9	Анализ и визуализация результатов сканирования	20
4.10	Бенчмаркинг с разными параметрами	24
4.11	Сохранение всех результатов	27
5	Выводы	29

List of Figures

3.1	Создание репозитория	7
3.2	Создание gpg ключа	8
3.3	Добавление ключа на GitHub	8
3.4	Установка необходимых пакетов	8
3.5	Клонирование репозитория	8
3.6	Установка необходимых пакетов	9
3.7	git flow	9
3.8	Установка Quarto	9
3.9	Установка Julia	10
3.10	первая работа в Julia	10
3.11	Результат test_setup.jl	10
3.12	Запуск 01_exponential_growth.jl	10
3.13	Создание литературных файлов	10

List of Tables

1 Цель работы

Создание среды для выполнения лабораторных работ по курсу имитационного моделирования.

2 Задание

- Создать рабочий каталог для всего курса.
- Создать рабочее пространство для программ в рамках лабораторной работы.
- Выполнить все задания по тексту лабораторной работы.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Выполнение лабораторной работы

Создаём репозиторий на основе шаблона(рис. 3.1).

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk ().*

Start with a template
Templates pre-configure your repository with files. yamadharma/course-directory-student-template

Include all branches
If enabled, all branches from the template repository will be included. Off

1 General

Owner * serapshi / **Repository name *** 2026-1--study--simulation- modeling

✔ Your new repository will be created as 2026-1--study--simulation--modeling. The repository name can only contain ASCII letters, digits, and the characters -, ., and _.

Great repository names are short and memorable. How about [legendary-fishstick](#)?

Description
Имитационное моделирование
26 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository. Public

[Create repository](#)

Figure 3.1: Создание репозитория

Генерация ключа для подписи коммитов.(рис. 3.2).

Добавление скопированного ключа на гитхаб(рис. 3.3).

Установка gh(рис. 3.4).

Клонирование репозитория на виртуальную машину(рис. 3.5).

Установка необходимых пакетов rpm(рис. 3.6).

```

sypavliuchenkov@Agnis:~/work/study/2026-1/2026-1==study-simulation-modeling$ gpg --full-generate-key
gpg (GnuPG) 2.4.0; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: directory '/home/sypavliuchenkov/.gnupg' created
gpg: keybox '/home/sypavliuchenkov/.gnupg/pubring.kbx' created
Please select what kind of key you want:
  (1) RSA and RSA
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) *default*
 (10) ECC (sign only)
 (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096
Requested keysize is 4096 bits
Please specify how long the key should be valid.
   0 = key does not expire
   <n> = key expires in n days
   <n>w = key expires in n weeks
   <n>m = key expires in n months
   <n>y = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: SVPavliuchenkov
Email address: thefrankleen@gmail.com
Comment:
You selected this USER-ID:
"SVPavliuchenkov <thefrankleen@gmail.com>"

```

Figure 3.2: Создание gpg ключа

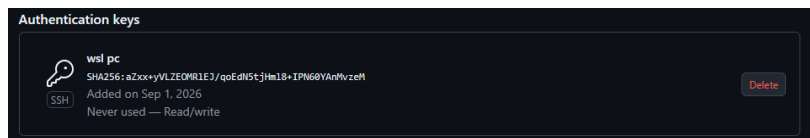


Figure 3.3: Добавление ключа на GitHub

```

sypavliuchenkov@Agnis:~/work/study/2026-1/2026-1==study-simulation-modeling$ sudo apt install gh
Installing:
  gh

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 2
  Download size: 8547 kB
  Space needed: 37.1 MB / 1023 GB available

Get:1 http://archive.ubuntu.com/ubuntu resolute/universe amd64 gh amd64 2.46.0-4 [8547 kB]
Fetched 8547 kB in 2s (3722 kB/s)
Selecting previously unselected package gh.
(Reading database ... 46477 files and directories currently installed.)
Preparing to unpack .../archives/gh_2.46.0-4_amd64.deb ...
Unpacking gh (2.46.0-4) ...
Setting up gh (2.46.0-4) ...
Processing triggers for man-db (2.13.1-1build1) ...
sypavliuchenkov@Agnis:~/work/study/2026-1/2026-1==study-simulation-modeling$ gh auth login
? What account do you want to log into? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: C629-4CE9
Press Enter to open github.com in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
! Authentication credentials saved in plain text
✓ Logged in as serapshi
sypavliuchenkov@Agnis:~/work/study/2026-1/2026-1==study-simulation-modeling$

```

Figure 3.4: Установка необходимых пакетов

```

sypavliuchenkov@Agnis:~/work/study/2026-1/2026-1==study-simulation-modeling$ gh repo clone serapshi/2026-1--study-simulation-modeling
Cloning into '2026-1--study-simulation-modeling'...
remote: Enumerating objects: 41, done.
remote: Counting objects: 100% (41/41), done.
remote: Compressing objects: 100% (39/39), done.
remote: Total 41 (delta 1), reused 29 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (41/41), 25.56 KiB | 608.00 KiB/s, done.
Resolving deltas: 100% (2/2), done.
sypavliuchenkov@Agnis:~/work/study/2026-1/2026-1==study-simulation-modeling$ ls
2026-1--study-simulation-modeling
CONDUCT.md CODE_OF_CONDUCT.md CONTRIBUTING.md COURSE LICENSE Makefile README.en.md README.git-Flow.md README.md SECURITY.md package.json template

```

Figure 3.5: Клонирование репозитория


```

using DrWatson
@quickactivate "project"
using DifferentialEquations
using Plots
using DataFrames

function exponential_growth!(du, u, p, t)
    α = p
    du[1] = α * u[1]
end

u0 = [1.0] # начальная популяция
α = 0.3 # скорость роста
tspan = (0.0, 10.0) # временной интервал
prob = ODEProblem(exponential_growth!, u0, tspan, α)
sol = solve(prob, Tsit5(), saveat=0.1)
plot(sol, label="u(t)", xlabel="Время t", ylabel="Популяция u",
title="Экспоненциальный рост (α = $α)", lw=2, legend=:topleft)
savefig(plotsdir("exponential_growth_α=$α.png"))
df = DataFrame(t=sol.t, u=first.(sol.u))
println("Первые 5 строк результатов:")
println(first(df, 5))
u_final = last(sol.u)[1]
doubling_time = log(2) / α
println("\nАналитическое время удвоения: ", round(doubling_time; digits=2))

```

Первые 5 строк результатов:

5×2 DataFrame

Row	t	u
	Float64	Float64
1		

1		0.0	1.0
2		0.1	1.03045
3		0.2	1.06184
4		0.3	1.09417
5		0.4	1.1275

Аналитическое время удвоения: 2.31

Создаем `01_exponential_growth.jl`, проделаем аналогичные действия и получаем.

4 Параметрическое исследование экспоненциального роста

4.1 Активация проекта и загрузка пакетов

ИЗМЕНЕНИЕ: Добавлен DrWatson для управления проектом и параметрами

```
using DrWatson
@quickactivate "project" # Активация проекта DrWatson
using DifferentialEquations
using DataFrames
using Plots
using JLD2
using BenchmarkTools
```

Установка каталогов

```
script_name = splitext(basename(PROGRAM_FILE))[1]
mkpath(plotsdir(script_name))
mkpath(datadir(script_name))
```

"/home/svpavliuchenkov/work/study/2026-1/2026-1==study-simulation-modeling/2026-1--study--s

4.2 Определение модели

Модель: $\frac{du}{dt} = \alpha \cdot u$

```
function exponential_growth!(du, u, p, t)
    α = p.α # **ИЗМЕНЕНИЕ:** Параметры теперь передаются как именованный кортеж
    du[1] = α * u[1]
end
```

exponential_growth! (generic function with 1 method)

4.3 Определение параметров в Dict

ОСНОВНОЕ ИЗМЕНЕНИЕ: Все параметры собраны в Dict для систематизации
Базовый набор параметров (один эксперимент)

```
base_params = Dict{
    :u0 => [1.0], # начальная популяция
    :α => 0.3, # скорость роста
    :tspan => (0.0, 10.0), # интервал времени
    :solver => Tsit5(), # метод решения
    :saveat => 0.1, # шаг сохранения результатов
    :experiment_name => "base_experiment"
}

println("Базовые параметры эксперимента:")
for (key, value) in base_params
    println(" $key = $value")
end
```

Базовые параметры эксперимента:

$\alpha = 0.3$

```

u0 = [1.0]
saveat = 0.1
solver = OrdinaryDiffEqTsit5.Tsit5{typeof(OrdinaryDiffEqCore.trivial_limiter!), typeof(Ord
experiment_name = base_experiment
tspan = (0.0, 10.0)

```

4.4 Функция-обертка для запуска одного эксперимента

ИСПРАВЛЕНИЕ: Возвращаем Dict со строковыми ключами

```

function run_single_experiment(params::Dict)
@unpack u0, α, tspan, solver, saveat = params
prob = ODEProblem(exponential_growth!, u0, tspan, (α=α,)) # Создаем и решаем задачу
sol = solve(prob, solver; saveat=saveat)
final_population = last(sol.u)[1] # Анализ результатов
doubling_time = log(2) / α
return Dict(
"solution" => sol,
"time_points" => sol.t,
"population_values" => first.(sol.u),
"final_population" => final_population,
"doubling_time" => doubling_time,
"parameters" => params # Сохраняем исходные параметры
) # Используем строки как ключи для совместимости с DrWatson
end

```

run_single_experiment (generic function with 1 method)

4.5 Запуск базового эксперимента

ИЗМЕНЕНИЕ: Используем `produce_or_load` для автоматического кэширования☒

```
data, path = produce_or_load(
    datadir(script_name, "single"), # Папка для сохранения
    base_params, # Параметры эксперимента
    run_single_experiment, # Функция для выполнения
    prefix = "exp_growth", # Префикс имени файла
    tag = false, # Не добавлять git-тег
    verbose = true
)

println("\nРезультаты базового эксперимента:")
println(" Финальная популяция: ", data["final_population"])
println(" Время удвоения: ", round(data["doubling_time"]; digits=2))
println(" Файл результатов: ", path)
```

Результаты базового эксперимента:

Финальная популяция: 20.08544851618676

Время удвоения: 2.31

Файл результатов: /home/svpavliuchenkov/work/study/2026-1/2026-1==study-simulation-modelin

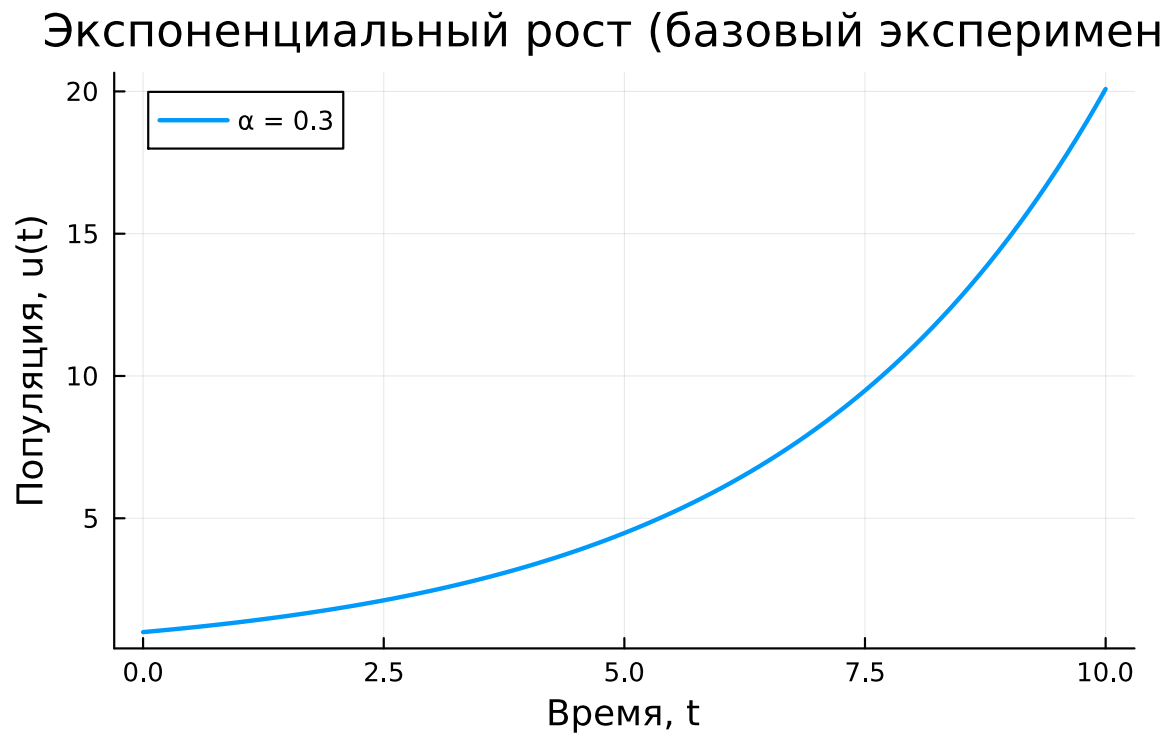
4.6 Визуализация базового эксперимента

```
p1 = plot(data["time_points"], data["population_values"],
    label="α = $(base_params[:α])",
    xlabel="Время, t",
    ylabel="Популяция, u(t)",
```

```

title="Экспоненциальный рост (базовый эксперимент)",
lw=2,
legend=:topleft,grid=true
)

```



Сохраним график в папку plots

```

savefig(plotsdir(script_name, "single_experiment.png"))

```

"/home/svpavliuchenkov/work/study/2026-1/2026-1==study-simulation-modeling/2026-1--study--s

4.7 Параметрическое сканирование

НОВАЯ СЕКЦИЯ: Исследование влияния параметра α Сетка параметров для сканирования

```

param_grid = Dict(
:u0 => [[1.0]], # фиксируем начальное условие
:α => [0.1, 0.3, 0.5, 0.8, 1.0], # исследуемые значения скорости роста
:tspan => [(0.0, 10.0)], # фиксируем интервал времени
:solver => [Tsit5()], # фиксируем метод решения
:saveat => [0.1], # фиксируем шаг сохранения
:experiment_name => ["parametric_scan"]
)

```

Dict{Symbol, Vector} with 6 entries:

```

:α          => [0.1, 0.3, 0.5, 0.8, 1.0]
:u0         => [[1.0]]
:saveat     => [0.1]
:solver     => [Tsit5{typeof(trivial_limiter!), typeof(trivial_limiter!)...}
:experiment_name => ["parametric_scan"]
:tspan      => [(0.0, 10.0)]

```

Генерация всех комбинаций параметров

```

all_params = dict_list(param_grid)
println("\n" * "="^60)
println("ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ")
println("Всего комбинаций параметров: ", length(all_params))
println("Исследуемые значения α: ", param_grid[:α])
println("="^60)

```

=====

ПАРАМЕТРИЧЕСКОЕ СКАНИРОВАНИЕ

Всего комбинаций параметров: 5

Исследуемые значения α : [0.1, 0.3, 0.5, 0.8, 1.0]

=====

4.8 Запуск всех экспериментов и сбор результатов

НОВАЯ СЕКЦИЯ: Автоматический запуск и сохранение всех вариантов

```
all_results = []
all_dfs = []
for (i, params) in enumerate(all_params)
println("Прогресс:  $i / \text{length}(\text{all\_params})$  |  $\alpha = \text{params}[\alpha]$ ")
data, path = produce_or_load(
datadir(script_name, "parametric_scan"), # Данные
params, # Текущий набор параметров
run_single_experiment, # Функция для выполнения
prefix = "scan", # Префикс имени файла
tag = false, verbose = false # Не выводить подробности для каждого запуска
) # Автоматическое сохранение/загрузка каждого эксперимента
result_summary = merge(
params,
Dict(
:final_population => data["final_population"],
:doubling_time => data["doubling_time"],
:filepath => path # Путь к сохраненным данным
)
) # Сохраняем сводные результаты (используем символы для параметров, но данные из data - строки)

push!(all_results, result_summary)
df = DataFrame(
```

```

t = data["time_points"],
u = data["population_values"],
α = fill(params[:α], length(data["time_points"]))
) # Сохраняем полные данные для визуализации
push!(all_dfs, df)
end

```

Прогресс: 1/5 | $\alpha = 0.1$

Прогресс: 2/5 | $\alpha = 0.3$

Прогресс: 3/5 | $\alpha = 0.5$

Прогресс: 4/5 | $\alpha = 0.8$

Прогресс: 5/5 | $\alpha = 1.0$

4.9 Анализ и визуализация результатов

сканирования

НОВАЯ СЕКЦИЯ: Сравнительный анализ всех экспериментов Сводная таблица результатов

```

results_df = DataFrame(all_results)
println("\nСводная таблица результатов:")
println(results_df[!, [:α, :final_population, :doubling_time]])

```

Сводная таблица результатов:

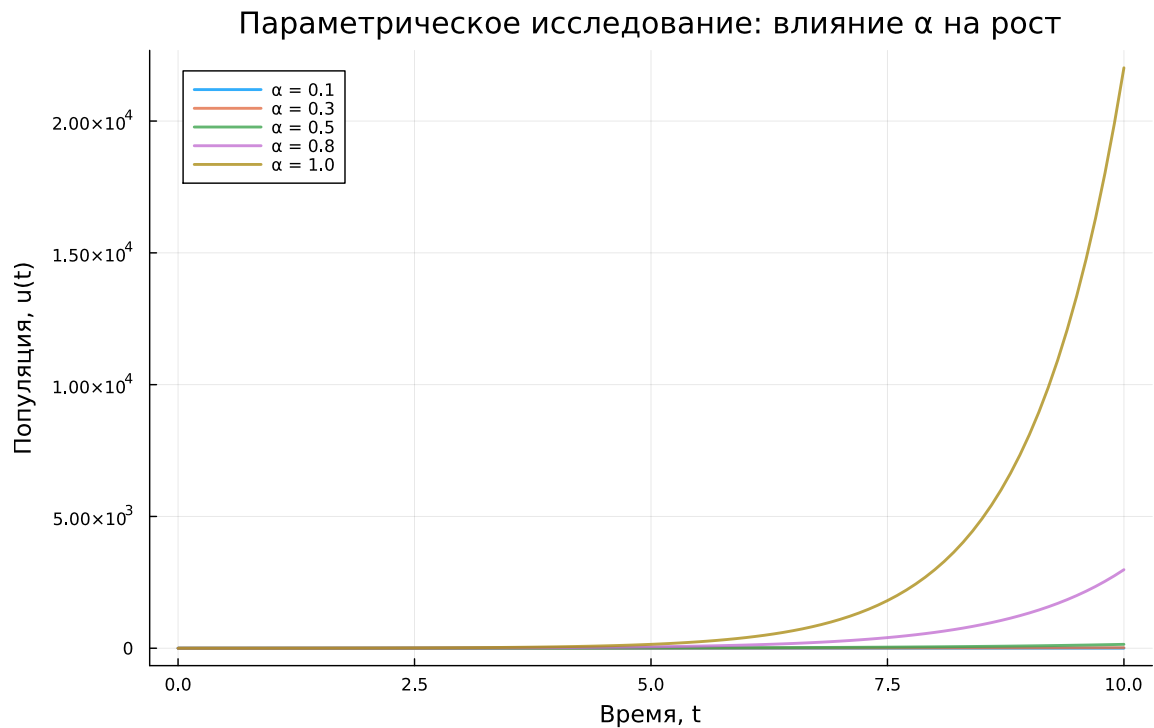
5×3 DataFrame

Row	α	final_population	doubling_time
	Float64	Float64	Float64

1	0.1	2.71828	6.93147
2	0.3	20.0854	2.31049
3	0.5	148.409	1.38629
4	0.8	2980.57	0.866434
5	1.0	22021.0	0.693147

Сравнительный график всех траекторий

```
p2 = plot(size=(800, 500), dpi=150)
for params in all_params
data, _ = produce_or_load(
datadir(script_name, "parametric_scan"),
params,
run_single_experiment,
prefix = "scan"
) # Загружаем данные (они уже есть на диске)
plot!(p2, data["time_points"], data["population_values"],
label="α = $(params[:α])",
lw=2,
alpha=0.8
)
end
plot!(p2,
xlabel="Время, t",
ylabel="Популяция, u(t)",
title="Параметрическое исследование: влияние α на рост",
legend=:topleft,
grid=true
)
```



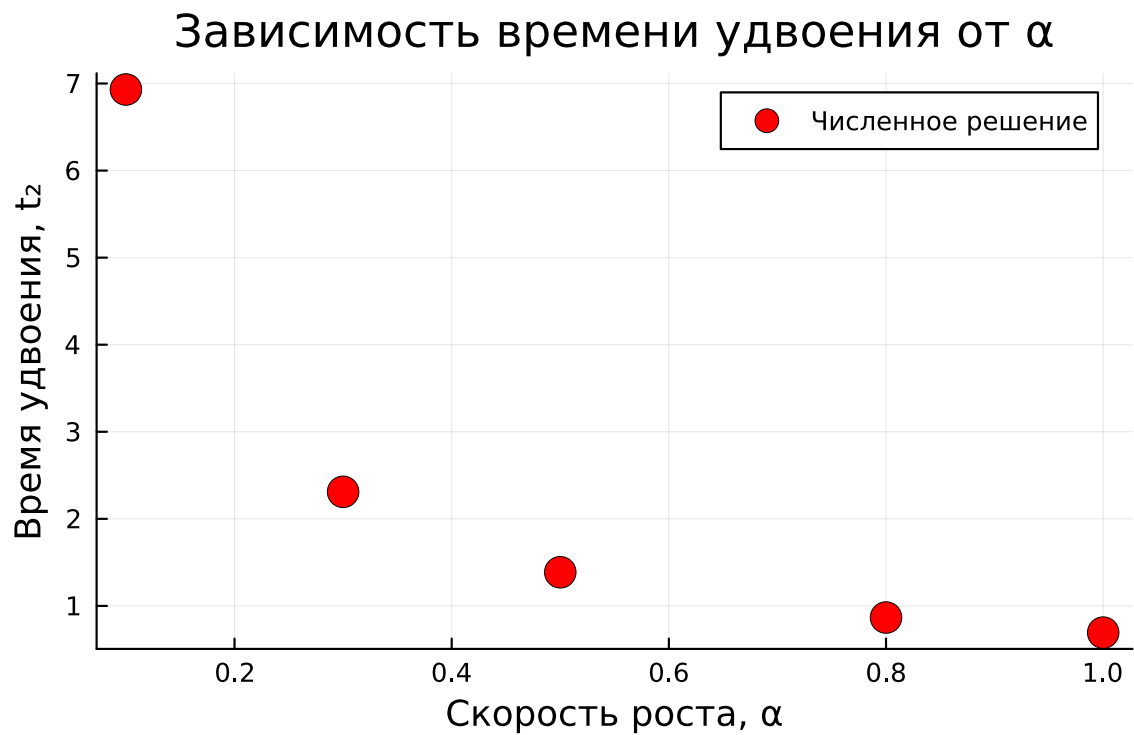
Сохраним график в папку plots

```
savefig(plotsdir(script_name, "parametric_scan_comparison.png"))
```

"/home/svpavliuchenkov/work/study/2026-1/2026-1==study-simulation-modeling/2026-1--study--s

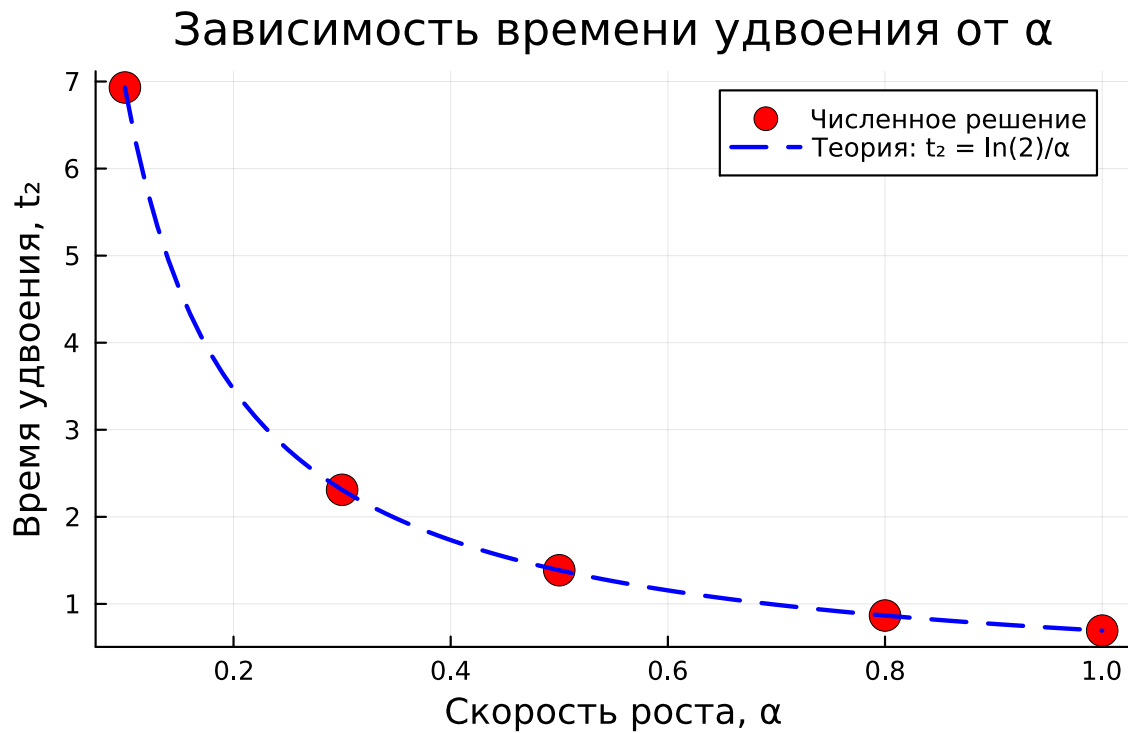
График зависимости времени удвоения от α

```
p3 = plot(results_df.alpha, results_df.doubling_time,
          seriestype=:scatter,
          label="Численное решение",
          xlabel="Скорость роста,  $\alpha$ ",
          ylabel="Время удвоения,  $t_2$ ",
          title="Зависимость времени удвоения от  $\alpha$ ",
          markersize=8,
          markercolor=:red,
          legend=:topright
        )
```



Теоретическая кривая: $t_2 = \ln(2)/\alpha$

```
alpha_range = 0.1:0.01:1.0
plot!(p3, alpha_range, log(2) ./ alpha_range,
label="Теория:  $t_2 = \ln(2)/\alpha$ ",
lw=2,
linestyle=:dash,
linecolor=:blue
)
```



Сохраним график в папку plots

```
savefig(plotsdir(script_name, "doubling_time_vs_alpha.png"))
```

"/home/svpavliuchenkov/work/study/2026-1/2026-1==study-simulation-modeling/2026-1--study--s

4.10 Бенчмаркинг с разными параметрами

ИЗМЕНЕНИЕ: Бенчмаркинг для разных значений α

```
println("\n" * "="^60)
println("Бенчмаркинг для разных значений  $\alpha$ ")
println("="^60)
benchmark_results = []
for  $\alpha$ _value in param_grid[: $\alpha$ ]
  bench_params = Dict(
    :u0 => [1.0],
```

```

:α => α_value,
:tspan => (0.0, 10.0),
:solver => Tsit5(),
:saveat => 0.1
) # Подготавливаем параметры для бенчмарка

function benchmark_run() # Функция для бенчмарка
prob = ODEProblem(exponential_growth!,
bench_params[:u0],
bench_params[:tspan],
(α=bench_params[:α],))
return solve(prob, bench_params[:solver];
saveat=bench_params[:saveat])
end

println("\nБенчмарк для α = $α_value:")
b = @benchmark $benchmark_run() samples=100 evals=1 # Запускбенчмарка↵
push!(benchmark_results, (α=α_value, time=median(b).time/1e9)) #время в секундах↵
println(" Среднее время: ", round(median(b).time/1e9;digits=4), " сек")
end

```

```

=====
Бенчмаркинг для разных значений α
=====

```

```

Бенчмарк для α = 0.1:
Среднее время: 0.0 сек

```

```

Бенчмарк для α = 0.3:
Среднее время: 0.0 сек

```

Бенчмарк для $\alpha = 0.5$:

Среднее время: 0.0 сек

Бенчмарк для $\alpha = 0.8$:

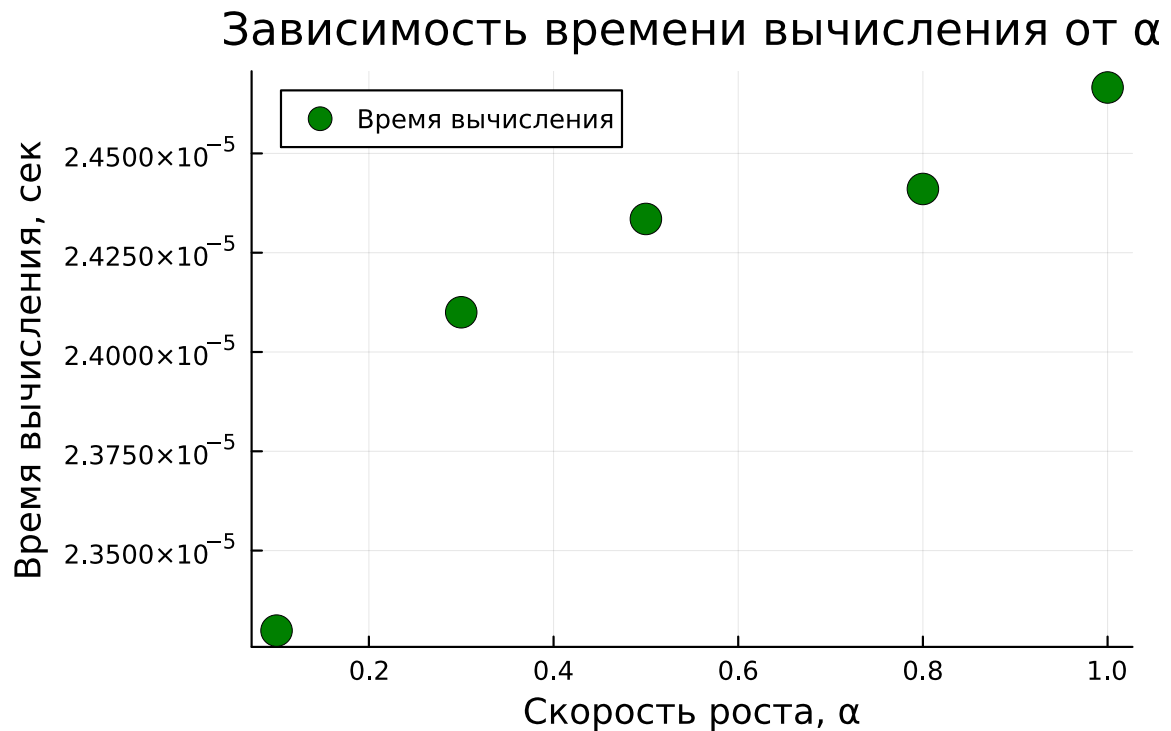
Среднее время: 0.0 сек

Бенчмарк для $\alpha = 1.0$:

Среднее время: 0.0 сек

График зависимости времени вычисления от α

```
bench_df = DataFrame(benchmark_results)
p4 = plot(bench_df.alpha, bench_df.time,
          seriestype=:scatter,
          label="Время вычисления",
          xlabel="Скорость роста,  $\alpha$ ",
          ylabel="Время вычисления, сек",
          title="Зависимость времени вычисления от  $\alpha$ ", markersize=8,
          markercolor=:green,
          legend=:topleft
        )
```



Сохраним график в папку plots

```
savefig(plotsdir(script_name, "computation_time_vs_alpha.png"))
```

```
"/home/svpavliuchenkov/work/study/2026-1/2026-1==study-simulation-modeling/2026-1--study--s
```

4.11 Сохранение всех результатов

НОВАЯ СЕКЦИЯ: Сохранение сводных данных для последующего анализа

```
@save datadir(script_name, "all_results.jld2") base_params param_grid all_params results_df
@save datadir(script_name, "all_plots.jld2") p1 p2 p3 p4

println("\n" * "="^60)
println("ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА")
println("="^60)

println("\nРезультаты сохранены в:")
println(" • data/$(script_name)/single/ - базовый эксперимент")
```

```
println(" • data/${script_name}/parametric_scan/ -параметрическое сканирование")
println(" • data/${script_name}/all_results.jld2 - сводныеданные")
println(" • plots/${script_name}/ - все графики")
println(" • data/${script_name}/all_plots.jld2 - объектыграфиков")
println("\nДля анализа результатов используйте:")
println(" using JLD2, DataFrames")
println(" @load \"data/${script_name}/all_results.jld2\"")
println(" println(results_df)")
```

=====

ЛАБОРАТОРНАЯ РАБОТА ЗАВЕРШЕНА

=====

Результаты сохранены в:

- data/startup/single/ - базовыйэксперимент
- data/startup/parametric_scan/ -параметрическое сканирование
- data/startup/all_results.jld2 - сводныеданные
- plots/startup/ - все графики
- data/startup/all_plots.jld2 - объектыграфиков

Для анализа результатов используйте:

```
using JLD2, DataFrames
@load "data/startup/all_results.jld2"
println(results_df)
```

5 Выводы

В ходе лабораторной работы было подготовлено рабочее пространство для выполнения программ и приобретены необходимые навыки создания и преобразования программ на Julia.